

INPUT DETERMINISM · RISK CONTROL

1. 为什么输入问题 会先于模型问题摧毁系统

最危险的不是挂掉，而是高自信地错。

这节课先把“输入确定性”从治理概念，立成企业 AI 系统的入口控制面。

开场定调

风险传播图

三条底线

静默失败

事故复盘

工程入口

Week02 学习链

01

Risk
为什么输入先坏

02

Inventory
从资源目录走到输入地图

03

Metadata / PII
把 provenance 与边界写清

04

Contract Gate
什么能进，什么必须拦

05

Manifest / Gate
把规则真正接进 ingest

这节课先解决什么问题

01 Risk

02 Inventory

03 Metadata

04 Contract

05 Manifest

为什么它重要

- 模型没换、Prompt 没大改、服务还在跑，但回答开始慢慢变歪。
- 上游 schema / 枚举漂移、定位字段缺失、PII 边界失守，都会先伤到系统。
- 如果把 AI 质量问题默认归因为模型问题，Week02 后续 4 讲都会被误读成附属治理动作。
- 这节课先立判断：系统最危险的状态不是报错，而是稳定地产生错误事实、错误引用、错误动作。

学完这节课，你至少应该能做到

- 解释为什么生产里的 AI 失真经常首先是输入问题，而不是模型问题。
- 用事实底线 / 证据底线 / 边界底线来描述输入风险。
- 说清楚输入问题如何穿透到检索、生成、工具和审计层。
- 在 OmniSupport Copilot 仓库里找到 Week02 的第一批工程入口。
- 跑过一次最小入口验证，确认 contract、manifest 和 tests 已经接上。

本课产出：你能把输入风险压成三条底线；能找到 contract / manifest / tests 的入口；能判断系统“变歪”时先从哪层查起。

先把 Week02 的地图看清

01 Risk

02 Inventory

03 Metadata

04 Contract

05 Manifest

风险意识

为什么输入先坏

对象建模

有哪些资源值得接

最小标准

metadata 与 PII 怎么统一

工程门禁

什么输入能放行/拦截

批次声明

这一次到底怎么接

本周一句话

把资产盘点、最小元数据、PII 分级、Data Contract 和采集策略，收口成一条可放行、可拦截、可审计的输入门禁链。

ticket

document

audio

video

统一案例线：Northstar 的三条产品线，最后都被压到四类主输入对象。

Lesson 01 在整门课里的位置

- 上承 Week01：接住系统蓝图、Done 边界与风险边界。
- 本课解决：为什么输入是否可信、可追溯、可合规，会先于模型问题摧毁系统。
- 下接 Lesson 02-05：asset inventory → metadata / PII → contract gate → manifest / gate。
- 再下接 Week03：把 contract 与 manifest 真正变成 ingest admission、state、report、replay。

PART 01

先立生产级判断

先把“最危险的状态”立住，再展开事实、证据与边界。

Lesson 01

输入确定性
不是治理补丁，
而是系统入口。

事实

证据

边界

KEY THESIS

生产里最危险的不是系统挂掉，而是系统还在跑，并且高自信地错

“系统没挂，并不代表它没坏。”

真正的危险是：
它还能跑、还能答、还能调用工具，
但事实、引用和动作已经开始稳定偏移。

错误事实

状态、时间窗、主键一漂，KPI 与 route 会一起偏。

错误引用

正文还在，但 page_no / bbox / section_path 一丢，证据链先断。

错误动作

边界与 policy 没前置，系统更容易越权检索或误动作。

Week02 不是“治理补丁周”，而是整门课的数据入口控制面。

RISK CHAIN

输入错误不是局部 bug：它会沿着 source → manifest → contract 一路穿透



如果输入侧没有被工程化，后面的每一层都只能在错误基础上继续加工。

DEMO VS PRODUCTION

同样是“答错”，Demo 和生产看到的根因经常完全不同

Demo

生产

最先看到的问题

输出不够聪明、格式不够稳

今天能答、明天变歪，但系统还在跑

团队最先修什么

Prompt、模型、reranker

输入事实、元数据、权限、边界

最容易忽略什么

上游 schema / metadata / PII 漂移

证据链与运行时 gate

真正先该修什么

常常被忽略

输入控制面

一句话收口：Demo 常常修下游表现；生产更该先修输入控制面，再谈表达优化。

静默失败会在“看起来正常”时继续积累风险

显式报错

团队感知快

- 问题会立刻暴露
- 影响明显
- 虽然疼，但可定位

静默失败

团队以为没问题

- 服务还能跑
- 回答偶尔还像是对的
- 工具也可能“正常执行”

最坏后果

- 错误事实：状态、时间窗、枚举漂移，统计与 route 一起偏。
- 错误引用：正文还在，但来源与定位信息断掉。
- 错误动作：边界没前置，越权检索与误动作都更隐蔽。

Lesson 01 的目标，就是让学生默认把“持续错但不报错”看成更高优先级的工程风险。

THREE BASELINES

把“数据质量不好”拆成三条可执行的底线：事实、证据、边界

事实底线

枚举 / 主键 / 时间窗 / 增量窗口

它说的是不是真的。

误解：字段没报错就算对

证据底线

doc_version / page_no / bbox /
section_path

我能不能回到来源。

误解：正文还在就够了

边界底线

PII 分级 / tenant / role / access policy

谁能看、谁能搜、谁能做。

误解：输出前再补一层审核就行

THREE SILENT FAILURES

一个 AI 支持系统最常见的三种静默失败

工单状态枚举变了

事实底线

表面：回答还能生成。

坏点：status 语义漂移。

后果：事实判断和动作一起偏。

文档证据字段没了

证据底线

表面：chunk 还能召回。

坏点：定位字段缺失。

后果：citation 与 tracing 先断。

音视频能读但边界失守

边界底线

表面：转写和字幕都能搜。

坏点：speaker_role / pii flag / frame_ts 没建模。

后果：越权、PII 暴露与误动作一起出现。

一句话收口：静默失败危险在于——系统没红灯，但团队已经在把错误当成正常输入继续消费。

PART 02

把风险讲成可复盘的事故

从语义漂移和 provenance 断裂，看“系统为什么看起来还能跑”。

Lesson 01

输入确定性
不是治理补丁，
而是系统入口。

事实

证据

边界

MISDIAGNOSIS

为什么很多团队会先修错地方：因为输出最显眼，输入最隐蔽

你先看到的现象

团队最常见的第一反应

更可能的根因

工单相关答案忽高忽低

调 Prompt / 换模型

状态枚举、时间语义或增量窗口漂移

文档召回了，但引用不稳

调 top-k / 调 rerank

metadata 与定位字段不足

某些用户能查到不该看到的内容

加一道输出审查

输入阶段 access / PII 边界没有前置

Tool 动作看似合理但结果经常错

增加 agent 推理链条

输入事实和工具边界没有被契约化

要点：不要一上来就修症状；先问“相同问题为什么会突然系统性偏”。

INCIDENT A · SEMANTIC DRIFT

最危险的不是字段没了，而是语义变了但系统仍然通过

01

上游把 pending
改成 in_progress

02

API 仍可返回
JSON 结构没坏

03

旧 contract
没有及时更新

04

RAG / KPI 继续
消费旧语义

05

tool route
动作依据被污染

屏幕上看起来像什么

- 服务没挂、字段还在
- contract 看起来还合法
- 回答偶尔还能对

实际已经坏在哪

- 语义已经漂移，运行时意义变了。
- KPI、检索解释和 tool route 会在旧语义上继续工作。
- 危险点：系统不是红灯，而是开始稳定地产生“看起来差不多”的错。

INCIDENT B · PROVENANCE BREAK

没有 provenance 的“正确答案”，在生产里往往不算合格答案

先看哪个字段先消失

- page_no 丢了：不能稳定指回原页
- section_path 丢了：结构上下文失真
- bbox 丢了：图表/局部内容无法准确定位
- doc_version 丢了：bad case 难以复现

一旦这些字段消失，回答可能仍然“像对的”，但引用、audit 和 bad case 复盘会先失效。

输入状态	回答能力	引用能力	审计能力
只有正文	可能还能答	很差	很差
正文 + page_no	基本可引用	较稳	较稳
再加 section_path + bbox + doc_version	可回指、可对比	稳	稳

结论：没有 provenance 的“正确答案”，在生产里通常不算合格答案。

DIAGNOSE FIRST

先做判断，再做动作：一旦看到“系统性漂移”，优先回输入层排查

01

只有少数问法出错

现象：问题集中在少数问法。
优先查：prompt / retrieval。

02

同样问题突然系统性偏

现象：一周前正常，今天开始变歪。
优先查：schema、枚举、时间窗、增量窗口。

03

citation 开始失效

现象：回答像对的，但引用不稳。
优先查：metadata / provenance。

04

tool 参数开始越界

现象：动作看似合理，但边界在漏。
优先查：policy / contract / access boundary。

判断口令：相同问题“突然系统性偏”，先查输入；只有少数问法出错，再看 prompt / retrieval。

PART 03

从行业信号走到工程入口

不是只在讲治理理念，而是在接官方实践、仓库对象和 Week03 起跑线。

Lesson 01

输入确定性
不是治理补丁，
而是系统入口。

事实

证据

边界

INDUSTRY SIGNALS

行业正在把输入边界、结构边界和来源边界前置成工程对象

Anthropic

simple, composable patterns

最成功的 agent 实现并不是靠复杂黑盒，而是靠简单、可组合的工作流。

OpenAI

Structured Outputs / schema

系统边界被当成可验证接口；JSON Schema 不再只是输出格式问题。

Azure AI Search

multimodal retrieval

顺序、布局、annotation 与全文索引需要一起工作，而不只是抽正文。

NIST

risk management upfront

生成式 AI 风险管理强调前置，而不是等上线后再被动补锅。

共识只有一句话：系统越要可靠，越不能把输入当成“先喂进去再说”的黑盒。

输入问题不是局部 bug：它会同时穿透到 RAG、Tool 与 Audit 三层

输入问题

schema / 枚举漂移

metadata 缺失

PII / access 边界缺失

RAG 层怎么坏

召回和解释口径变偏

命中有了但 citation 不稳

检索越权

Tool 层怎么坏

路由与动作依据失真

工具拿不到必要上下文

工具越权

Audit 层怎么坏

事后很难定位到底哪批数据出了问题

报告里无法回到原页、原段、原时间

合规与追责一起失效

1

盘点资产

2

定义元数据 / PII

3

Data Contract 做成门禁

4

Manifest 接进
ingest

Week02 不是写文档结束：contract、manifest 与 tests 会真的进入 pipeline

contract

四类 data contract

manifest

source manifest / schema

dry-run

seed_loader

tests

pytest tests/contract/ -v

验证 1

看四类 contracts/data/*.json 是否已经在守事实、证据和边界。

验证 2

看 source_manifest_schema.json 与三份样例 manifest，确认 manifest 不是“文件列表”。

验证 3

跑一遍 seed_loader dry-run 与 tests/contract/，确认 contract 已经进入工程门禁。

真实终端里最短要跑什么

```
python -m pipelines.ingestion.seed_loader ...
```

```
pytest tests/contract/ -v
```

注意：本页只告诉学生“接下来验证什么”；命令不要在网页里逐字念。

RECAP & NEXT

这节课真正完成了什么：先把输入控制面立住，再把 Week02 的工程入口接上

本课最重要的 7 个判断（收口版）

- 生产里更常见的是输入先坏，而不是模型先坏。
- 输入风险至少要分成事实底线、证据底线、边界底线三类。
- 只修输出层，经常是在修症状，不是在修根因。
- Manifest 不是文件清单，而是一次 ingest 的声明入口。
- Contract 不是文档说明，而是运行时门禁。
- metadata 与 PII 不是附加字段，而是 citation、filter、audit 的基础设施。
- Week02 做得越扎实，Week03 之后的 ingest、RAG、Agent、评测与治理越稳。

进入 Lesson 02 之前，先做 3 件事

- 把四类 contract 和三份 manifest 再打开一遍，先建立地图感。
- 跑一遍 seed_loader 与 tests/contract/，确认入口不是空概念。
- 写下你最担心的一类输入风险：事实、证据，还是边界。

下一讲

从资源目录到输入地图

Lesson 02 会把 “
有哪些输入对象值得接入系统” 真正盘成资产地图。

ASSET INVENTORY · INPUT MAP

2. 从资源目录到输入地图

盘点不是列目录；而是把企业数据源变成可准入、可追溯、可消费的输入对象。

对象建模

四问资格审查

四类资产

分层准入

工程入口

授课口径：学生讲义页主导，教师页风格统一；这讲要把“资源”压成“系统可消费对象”。

Week02 学习链

01

Risk
为什么输入先坏

02

Inventory
从资源目录走到输入地图

03

Metadata / PII
把 provenance 与边界写清

04

Contract Gate
什么能进，什么必须拦

05

Manifest / Gate
把规则真正接进 ingest

这节课先解决什么问题

01 Risk

02 Inventory

03 Metadata

04 Contract

05 Manifest

为什么它重要

- 企业数据源不是一类东西：工单、文档、音频、视频的“可消费条件”完全不同。
- 只列目录会漏掉 update window、权限边界、证据定位和 owner 责任。
- 一旦资源没有准入状态，后续 contract / manifest / ingest 都会变成补丁。
- Lesson02 是把“风险意识”转成“输入对象地图”的第一步。

学完这节课，你至少应该能做到

- 区分 source、input asset、serving object，避免把源系统、输入对象和服务对象混在一起。
- 用事实 / 证据 / 边界 / 责任四问做资格审查。
- 为 ticket、document、audio、video 四类主资产定义最小盘点字段。
- 给每个资产打上 ready_now / conditional / hold / exclude 的准入状态。

本课产出：一份可让 metadata minimum、PII matrix、contract gate 与 manifest 接住的资产清单。

先把 Week02 的地图看清

01 Risk

02 Inventory

03 Metadata

04 Contract

05 Manifest

01 风险意识

为什么输入先坏

02 对象建模

有哪些资源值得接

03 最小标准

metadata 与 PII 怎么统一

04 工程门禁

什么输入能放行/拦截

05 批次声明

这一次到底怎么接

本周一句话

把资产盘点、最小元数据、PII 分级、Data Contract 和采集策略，收口成一条可放行、可拦截、可审计的输入门禁链。

ticket

document

audio

video

Lesson 02 在整门课里的位置

- 上承 Lesson01：三条底线不是口号，而是盘点时的资格问题。
- 本课解决：哪些输入对象值得进入系统，以及以什么资格进入。
- 下接 Lesson03：资产能否进入运行时，取决于 metadata minimum 与 PII policy 是否足够清楚。

PART 01

先把资源看成对象

资源只有被建模成输入对象，后面才谈得上 metadata、PII、contract 与 manifest。

Lesson 02

盘点不是列目录，
而是输入对象建模。

事实

证据

边界

责任

KEY THESIS

资产盘点不是把文件列全，而是回答：哪些输入对象值得进入系统

**“资源很多”不是能力，
“输入对象可控”才是能力。**

真正要盘的是：这个资源从哪来、谁负责、以什么频率更新、能否定位证据、谁能看、能否被下游稳定消费。

可发现

Discoverable

系统知道它在哪里、来自谁、怎么更新。

可判断

Qualifiable

系统能判断它是否满足事实、证据、边界与责任要求。

可消费

Consumable

系统知道它会变成 RAG chunk、KPI mart、tool input 还是 audit evidence。

一句话收口：盘点不是“有什么”，而是“能不能带着资格进入系统”。

LECTURE DIAGRAM · REDRAW

讲义核心图：source → input asset → serving object



source 是“资源从哪来”；input asset 是“进入系统前怎么被命名与审查”；serving object 是“运行时到底被谁消费”。

THREE OBJECTS

三种对象别混：资源目录、输入地图、服务对象

资源目录

Resource Catalog

回答：哪里有什么资源。

例：某个 S3 bucket、有一套帮助中心、有一批客服工单。

常见坏法：把它当成同一个 Excel 表随便填。

输入地图

Input Map

回答：哪些资源能以什么资格进系统。

例：doc_asset_v3 可进 RAG；audio_call 只能 conditional。

常见坏法：把它当成同一个 Excel 表随便填。

服务对象

Serving Object

回答：最终被哪个运行时能力消费。

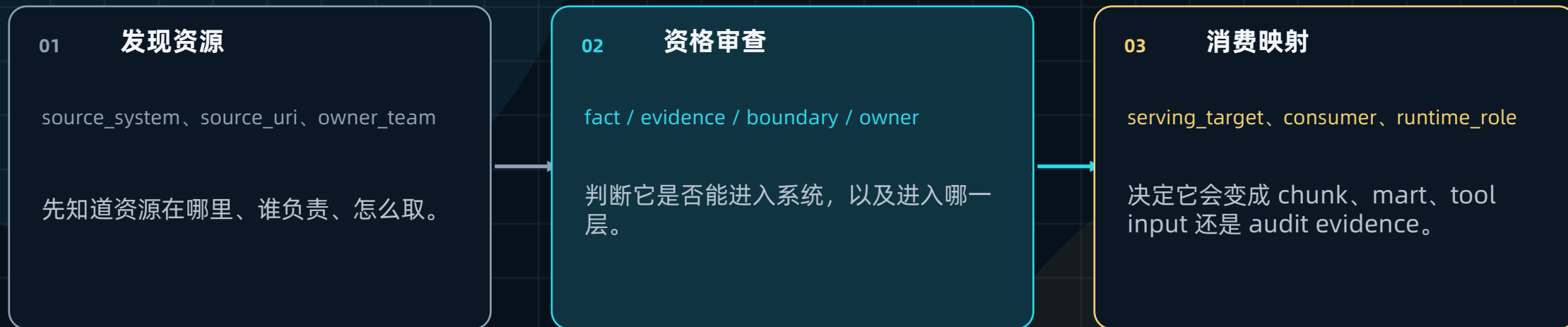
例：chunk 给检索；ticket_fact 给 KPI；tool_input_view 给工具。

常见坏法：把它当成同一个 Excel 表随便填。

如果一个字段既像源系统字段、又像运行时字段，先停下来问它到底属于哪一层。

PROCESS

从发现资源 → 资格审查 → 消费映射



输出不只是 inventory.csv，而是三类后续对象的入口：metadata minimum、contract tests、source manifest。

如果第二步资格审查缺失，第三步消费映射就会把不合格资源推给下游。

资格审查四问：事实、证据、边界、责任

事实

Fact

它说的事实能不能被稳定解释？

缺了：枚举、时间窗、主键语义一漂，下游统计与动作一起偏。

证据

Evidence

它能不能回到原始来源？

缺了：citation、bad case 复盘和审计会先断。

边界

Boundary

谁能看、谁能搜、谁能传工具？

缺了：越权检索、PII 暴露和误动作会变隐蔽。

责任

Owner

谁负责更新、修复和解释口径？

缺了：上游一变，没人知道该改 contract 还是 manifest。

四问不是治理语言，而是工程门禁的前置输入。

INVENTORY MINIMUMS

盘点表四组字段：不要让 inventory 退化成“三列清单”

Identity`asset_id / asset_class / asset_name / product_line`

先把对象命名清楚，避免 source 与 serving object 混写。

Source & Freshness`source_system / source_uri / refresh_policy / update_window`

说明资源从哪来、怎么变、多久更新一次。

Evidence & Lineage`evidence_locator / source_fingerprint / doc_version / ingest_batch_id`

让后续 citation、bad case 和 lineage 能回到来源。

Governance & Admission`owner_team / pii_level / access_policy / admission_status / quality_notes`

说明谁负责、谁能看、能不能进入系统。

课堂提醒：字段不要为了“显得完整”而堆；每一个字段都要能回答一个运行时问题。

ANTI-PATTERNS

三种假动作：看起来在盘点，实际上没有形成输入地图

Fake move

只列资源目录

我们有 200 份 PDF、20 张表、30 段视频

问题：没有可准入判断，也没有下游消费映射。

修正：加 owner、freshness、evidence_locator、admission_status。

Fake move

只写字段说明

每列都有 description，所以应该够了

问题：解释了字段长相，但没解释是否能进系统。

修正：字段说明 + 资格审查 + 质量/边界备注。

Fake move

不写准入状态

“先都接进来，后面再清洗

问题：把源头不确定性推给 chunk、retrieval 和 tool。

修正：ready_now / conditional / hold / exclude 前置。

只有资源在哪里不叫输入地图；能说明“是否能进、进哪里、谁负责才叫输入地图。

PART 02

四类主输入资产怎么盘

同一个 inventory 方法，要能覆盖结构化工单、文档、音频和视频四类资产。

Lesson 02

盘点不是列目录，
而是输入对象建模。

事实

证据

边界

责任

ASSET CARD · TICKET

Ticket 资产：最怕字段还在，但状态、时间和租户语义已经漂了

重点列

ticket_id

event_time

status / priority

tenant_id

product_line

owner_team

update_cursor

盘点时先问：status 有没有业务定义？event_time 是发生时间、更新时间还是入湖时间？tenant_id 能否成为权限过滤字段？

入门判断：ticket 资产可以 ready_now 的前提，是状态语义、时间语义、tenant 边界和 owner 都明确。

最常见误判

- 把 status 当作普通枚举，不记录 closed / resolved / waiting_vendor 的业务差异。
- 只看 updated_at，不区分 event_time 与 ingestion_time。
- 只接 ticket 表，不接 comment / event，导致过程证据缺失。
- 忘记 tenant / role / owner，后续 tool 调用无法边界控制。

如果结构没坏但口径变了，这就是 Lesson01 的“高自信地错”。

Document 资产：不要只抽正文，必须把版本、页码、章节和坐标留下来

重点列

doc_id

doc_version

source_fingerprint

page_no

section_path

bbox

license_tag

effective_date

盘点时先问：这份文档的版本怎么变化？证据能回到哪一页、哪一节、哪个 bbox？license 是否允许进入训练 / 检索 / 展示？

文档最小盘点不是“PDF 名称 + 文本”，而是 provenance 的入口。

最常见误判

- 只做 OCR / parse，把 page_no、bbox、section_path 丢掉。
- 多个版本文档混在一起，回答时无法复现来源。
- 将 license / entitlement 视为后处理，而不是输入边界。
- 把手册、FAQ、Release Notes 当成同一种文档，不区分消费场景。

文档盘点的关键不是“能读”，而是“能回指”。

Audio 资产：转写不是终点，说话人、时间戳、置信度和脱敏状态才是入口

重点列

call_id

transcript_id

speaker_role

confidence

pii_redaction_flag

consent_tag

start_ts / end_ts

盘点时先问：谁在说？说到哪个时间点？ASR 置信度够不够？PII 是否已经脱敏或标记？

Audio 不是文本的另一种来源；它天然带有 speaker、time 与 privacy 上下文。

最常见误判

- 把 transcript 当普通文档，不保留 speaker_role 与 start_ts。
- 只存完整通话文本，无法定位到某个话轮或片段。
- 不记录 ASR 置信度，导致低质量片段进入检索。
- PII 脱敏状态不清，下游很难决定能否索引、展示或传工具。

音频资产最容易丢的是“上下文边界”，不是转写文本本身。

Video 资产：不能只把视频当成“带画面的音频”

重点列

video_id

segment_ts

frame_ts

transcript_ref

image_caption

ocr_text

release_id

license_tag

盘点时先问：片段边界怎么切？关键帧如何定位？画面里的 OCR / 图示 / 操作步骤如何回指到原视频？

Video 的 serving object 往往不是一个：transcript chunk、image caption、OCR token、frame evidence 都可能分开被消费。

最常见误判

- 只保留视频转写，不保留 frame_ts 与关键帧证据。
- OCR 与 caption 进入同一个文本池，导致来源和置信度混乱。
- 没有 release_id / license_tag，后续发布和权限很难治理。
- 不区分教程步骤、错误日志、界面截图和旁白内容。

视频盘点是多模态输入地图的压力测试。

ADMISSION LADDER

分层准入：不是所有资产都应该 ready_now

ready_now**可直接进入下一步**

字段齐、owner 明确、证据定位可用、PII / access 可判断。

conditional**带条件进入**

可试运行，但必须带 quality_notes / risk_notes / 后续补齐项。

hold**暂缓进入**

关键信息缺失：证据定位、权限、版本或更新窗口不清。

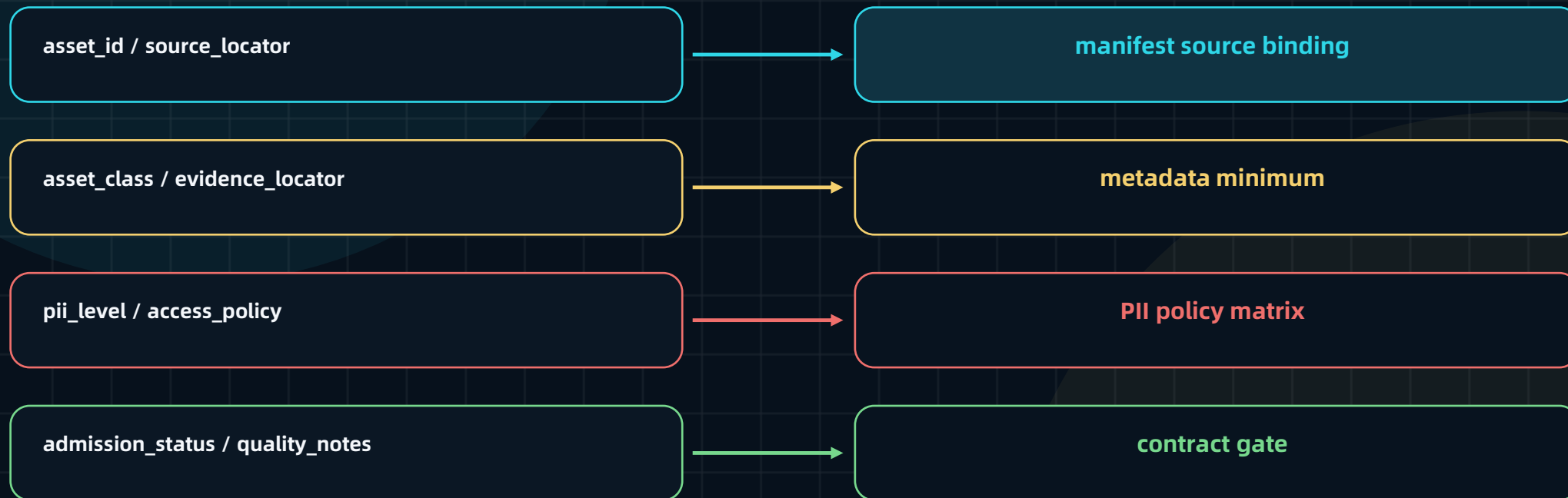
exclude**明确排除**

不合规、不可授权、价值低或不该被 AI 系统消费。

准入状态不是主观标签，而是后续 contract gate / manifest / seed loader dry-run 的前置判断。

DOWNSTREAM BINDING

从 inventory 到 metadata、PII、contract、manifest



不要把 inventory 看成一次性文档；它是 Week02 后四个对象的第一张事实表。

INDUSTRY SIGNALS

行业正在把“资产对象、元数据、血缘、模态顺序”前置成工程对象

这一页不是工具介绍会，而是让学生看到：输入地图是主流工程方向。

OpenMetadata

asset details / data quality /
lineage

数据资产页面顶层显示 Source、Owner、Tier、Type、Usage、Description，并提供 Schema、Profiler、Lineage 等 tabs。

DataHub

entity / aspect / relationship

metadata 以 Entity、Aspect、Relationship 建模，ownership、tags、glossaryTerms 等作为可治理方面。

OpenLineage

dataset / job / run events

以 dataset、job、run 和 facets 记录运行时 lineage，帮助定位 root cause 与影响范围。

Azure AI Search

multimodal search pipeline

多模态搜索把 text、image、video、audio 纳入同一检索思路，并强调顺序、hybrid query 与来源回指。

共识只有一句：生产级 AI 的输入，不是黑盒原料，而是带身份、责任、边界和血缘的资产对象。

现在切到本地：验证 inventory 不是空概念

01 asset inventory

data/asset_inventory/week02_
asset_inventory.csv

02 contracts

contracts/data/*.json

03 seed manifests

data/seed_manifests/*.yaml

04 tests

tests/inventory/test_asset_inve
ntory.py

本地验证 3 件事

- 列头是否覆盖 identity / freshness / evidence / governance 四组字段。
- 四类样例是否都能说明 source、locator、owner、PII 与 admission_status。
- ready_now / conditional / hold / exclude 是否能被测试识别。

真实终端里最短要跑什么

```
head -n 5  
data/asset_inventory/week02_asset_inventory.csv  
pytest tests/inventory/ -v
```


CSV WALKTHROUGH

CSV 最小样例：先看三列，再看整张表

```
asset_id,asset_class,source_system,evidence_locator,pii_level,admission_status,serving_target  
tkr.events,ticket,zendesk,ticket_id+event_time,internal,ready_now  
,kpi/rag/tool  
doc.edge_manual,document,minio,doc_version+page_no+section_path+box,public,ready_now,rag  
aud.call_0420,audio,minio,call_id+speaker_role+start_ts,restricted,conditional,rag/audit  
vid.tutorial_07,video,minio,video_id+segment_ts+frame_ts,internal,hold,routing
```

讲解顺序

- 1 **asset_class**
四类对象不能混。
- 2 **evidence_locator**
没有定位字段，就没有证据链。
- 3 **admission_status**
不是所有资产都 ready。

这不是 CSV 教学；这是一张把资源转成输入对象的最小地图。

RECAP & NEXT

这节课真正完成了什么：把资源盘点升级成输入对象地图

本课最重要的 7 个判断

- 资产盘点不是列目录，而是对象建模。
- source、input asset、serving object 必须分层。
- 资格审查至少问事实、证据、边界、责任。
- ticket、document、audio、video 四类资产的最小字段不同。
- admission_status 是后续门禁的前置输入。
- inventory 会喂给 metadata、PII matrix、contract、manifest。
- 没有 owner / evidence / boundary 的资源，不应该默认 ready。

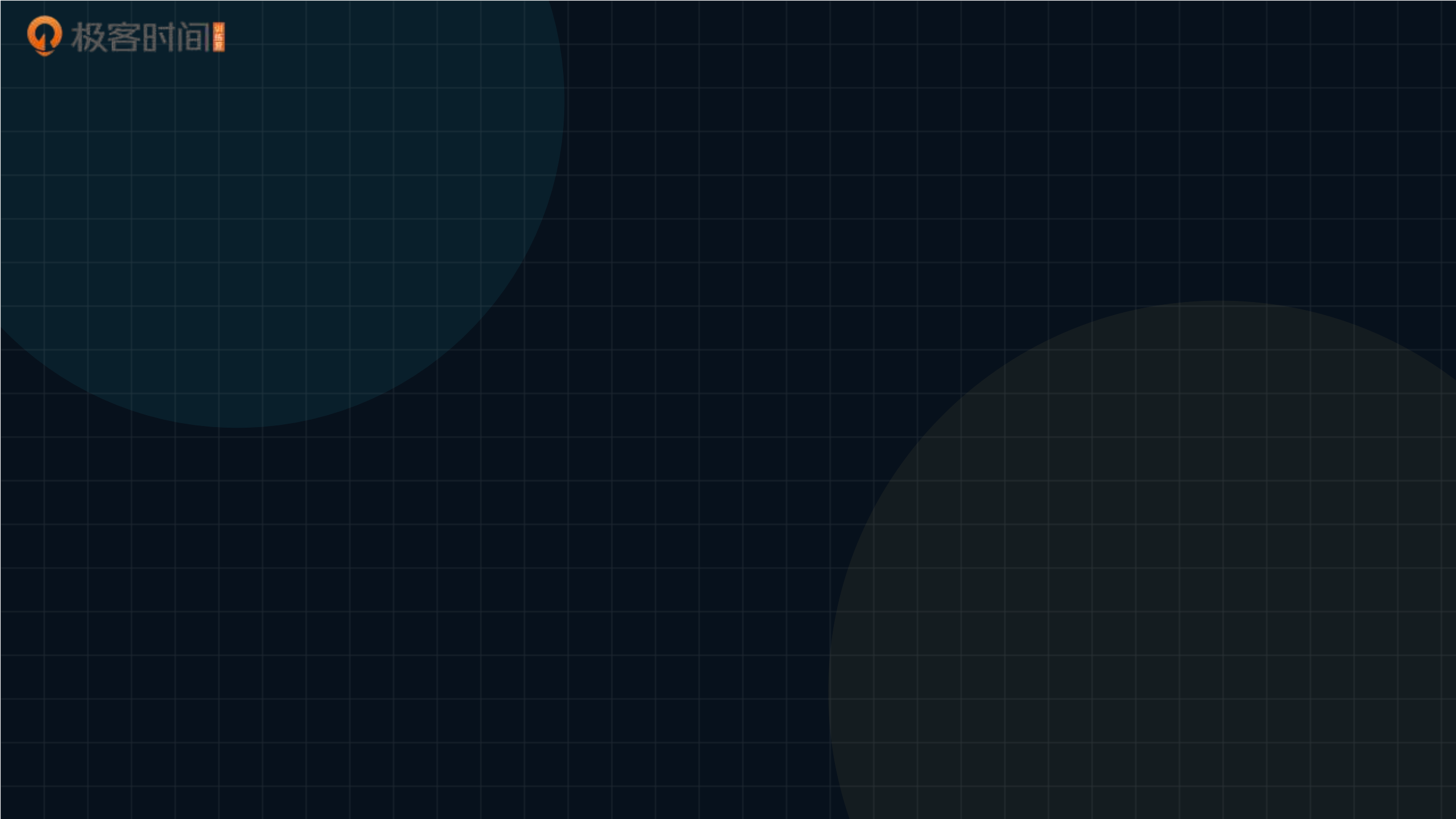
进入 Lesson 03 之前，先做 3 件事

- 打开 inventory，把四类资产各找一条样例。
- 标出每条样例缺的是事实、证据、边界还是责任。
- 写下哪些字段必须进入 metadata minimum。

下一讲

多模态最小元数据 与 PII 分级

Lesson 03 会把今天的输入地图，变成运行时可消费的 metadata minimum 和 PII policy matrix。



WEEK02 · INPUT DETERMINISM

多模态最小元数据 与 PII 分级

文档、音频、视频、工单怎么统一成可检索、可引用、可审计的 runtime interface

本课交付

01

metadata_minimums_v1.md

共享核心 + 四类模态扩展

02

pii_policy_matrix_v1.csv

字段级分级 + 动作矩阵

03

sample_records.json

给 Lesson04 contract tests 的样例

这节课解决什么问题

从“有资产清单”到“系统知道怎么消费这条输入”。

FROM L2

上一讲已经有了

docs/blueprints/week02/asset_inventory_v1.csv

但它只说明“我们有哪些资产”，不能直接保证检索、引用、权限和审计稳定。

RULE 1

最低上下文

来自哪里？能否回指？谁能消费？

RULE 2

PII 动作

哪些字段要脱敏、隔离、拒绝或人工审核？

OUTPUT

本课结果

metadata_minimums_v1.md
pii_policy_matrix_v1.csv
sample_records.json

这些不是说明文，而是后续 gate 会消费的规则。

学完你应该能做到

OUTCOME 1

解释证据链字段为什么是硬约束

page_no / bbox / section_path / start_ts / speaker_role / frame_ts

OUTCOME 2

定义四类资产的最小 metadata

ticket / document / audio / video: 共享核心 + 模态扩展

OUTCOME 3

把 PII 从 bool 变成动作矩阵

detect / store_raw / embed / retrieve / display / pass_to_tool / human_review

OUTCOME 4

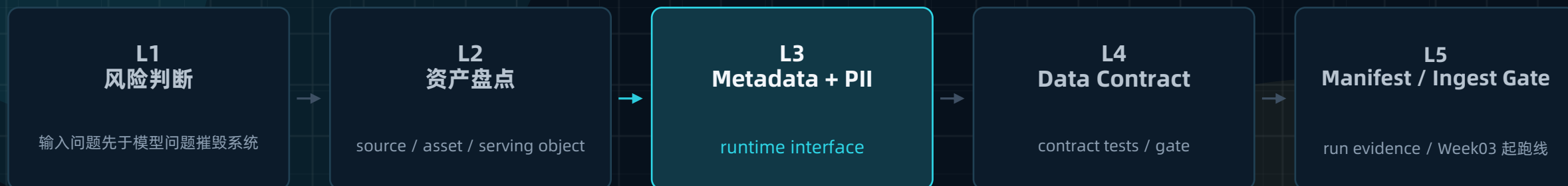
在 repo 中沉淀可执行工件

metadata_minimums_v1.md + pii_policy_matrix_v1.csv + sample_records.json

这不是“字段更多”，而是系统第一次知道一条输入能不能被引用、送模、展示、追责。

Week02 的位置：Lesson03 是中间层

它把资产清单转成 contract / manifest / gate 可以消费的上下文。



从 L2 接过来

asset_inventory_v1.csv 说明“这是什么资源、能不能接入”。

交给 L4 / L5

sample_records.json + PII matrix 让 contract 和 manifest 有可验证对象。

CORE THESIS

metadata 不是备注， 而是检索、引用、权限和审计的 runtime interface

缺 metadata 的系统，常常不是“答不了”，而是“答了也无法证明、无法过滤、无法追责”。

retrieval
过滤边界

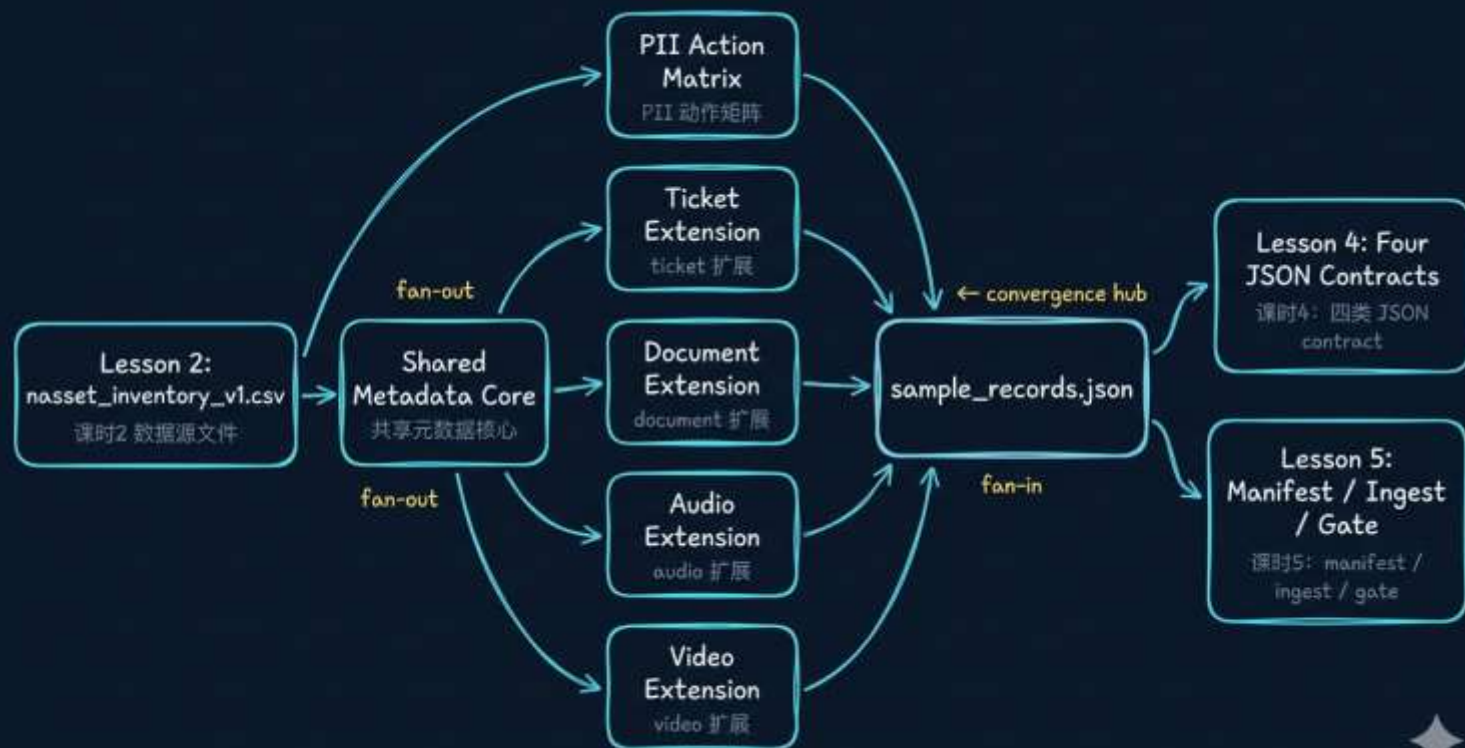
citation
证据回指

policy
动作门禁

audit
追责定位

先看总图：metadata 在运行时被谁消费

图来自学生讲义页：L2 的资产清单 fan-out 成共享核心和四类扩展，再汇入 sample_records.json。



convergence hub: sample_records.json

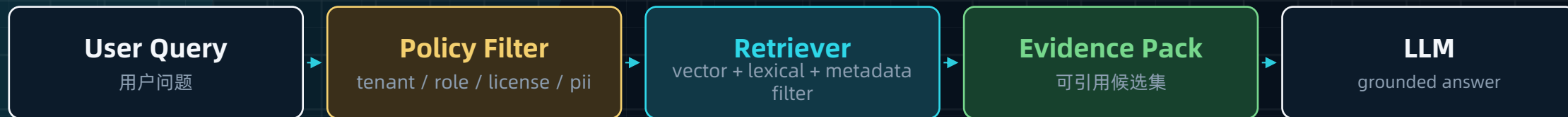
metadata 到底在运行时被谁消费

每个字段都要能回答：谁用它？影响什么决策？

metadata 字段	谁消费它	直接影响
access_scope	retrieval / tool layer	权限过滤：谁能搜、谁能触发工具动作
page_no / bbox / section_path	citation / audit	引用能不能回到原文页、区域、章节
speaker_role / start_ts / end_ts	transcript QA / HITL / review	对话责任、片段定位、人工复核入口
schema_version	contract / compatibility check	契约演进感知：新增字段还是 breaking change
pii_level	policy engine	是否可入模、可展示、可传工具

为什么 metadata 会直接决定检索过滤

生产系统不能“先召回所有内容，再祈祷模型别泄露”。



access_scope

缺失时：过滤失效，可能越权召回。

tenant_id

缺失时：多租户边界被打穿。

product_line

缺失时：错产品线的文档/工单混入。

license_tag

缺失时：不应展示的内容被拿来回答。

过滤字段必须在 chunk / embedding / retrieval 之前进入 runtime metadata。

为什么 metadata 会直接决定引用与追责

很多系统“看起来能答”，但不具备可追责能力。

DOCUMENT

文档证据锚点

doc_version
page_no
section_path
bbox

AUDIO

音频 / 对话证据锚点

speaker_role
start_ts / end_ts
confidence
call_id

VIDEO

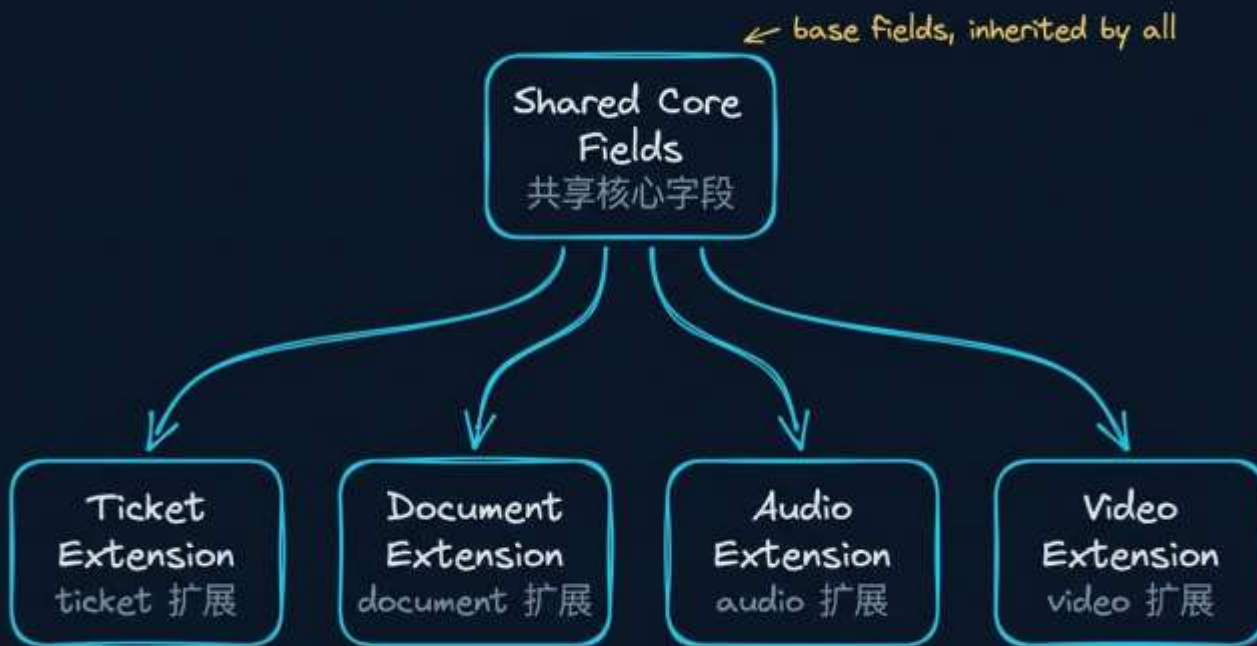
视频证据锚点

segment_ts
frame_ts
transcript_ref
ocr_text

危险状态：回答听起来像对的，但没人能证明它到底来自哪里。

统一方法：shared core + modality extension

所有资产继承一组共享核心字段，再按模态补充最小扩展字段。



共享核心字段：所有输入都必须继承

先把身份、版本、治理、时间放到同一层，后面才能统一 gate。

Identity

source_id

asset_type

source_system

Version & Trace

source_fingerprint

schema_version

Governance

owner

access_scope

pii_level

Time

observed_at

经验判断：共享核心字段越早统一，后面 contract / manifest / audit 的补丁越少。

为什么最小 metadata 必须先于 chunking

切块再精致，如果没有上下文，也只是“看起来像从这里来的”。

ANTI-PATTERN

错误顺序

raw text → chunk → embedding

后果：chunk 命中以后，才发现不知道页码、章节、坐标、权限、PII 动作。

PRODUCTION

正确顺序

raw asset → shared core → modality extension → chunk / segment
→ gate

结果：每个片段都天然带证据锚点和策略上下文。

如果没有 page_no / bbox / section_path / start_ts / frame_ts，后面只能“像是从这里来的”。

四类输入的最小 metadata 清单

共享核心之外，每类资产还必须补齐能支撑自己场景的证据字段。

ASSET

ticket

ticket_id / tenant_id
status / opened_at / updated_at
requester_role / product_line

状态语义和增量判断容易漂

ASSET

audio

call_id / speaker_role
start_ts / end_ts / confidence
pii_redaction_flag / transcript_text

对话责任和脱敏状态不明确

ASSET

document

doc_id / doc_version
page_no / bbox / section_path
license_tag

回指不稳、citation 失真

ASSET

video

video_id / segment_ts / frame_ts
transcript_ref / image_caption
ocr_text

视频时序和视觉证据丢失

JSON 对照 1: document 不是只有 text

文档证据链的关键是版本、页码、坐标、章节和许可边界。

BAD SAMPLE

```
{
  "asset_type": "document",
  "doc_id": "DOC-337",
  "text": "升级失败后请重启设备。"
}
```



GOOD SAMPLE · evidence anchor

```
{
  "source_id": "knowledge_doc",
  "asset_type": "document",
  "doc_version": "v3.2",
  "page_no": 37,
  "bbox": [72, 128, 488, 220],
  "section_path": "升级/失败恢复",
  "license_tag": "course-safe"
}
```

page_no

bbox

section_path

license_tag

缺这些字段: citation 会失真, entitlement 会失控, 审计无法回指原文位置。

JSON 对照 2: audio 必须能定位话轮和片段

音频的风险不只在文本内容，还在说话人、时间片段和脱敏状态。

BAD SAMPLE

```
{
  "asset_type": "audio",
  "call_id": "CALL-445",
  "transcript_text": "My phone is 13800138000..."
}
```



GOOD SAMPLE · reviewable segment

```
{
  "source_id": "support_call",
  "asset_type": "audio",
  "speaker_role": "customer",
  "start_ts": "00:03:12",
  "end_ts": "00:03:40",
  "confidence": 0.94,
  "pii_level": "sensitive",
  "pii_redaction_flag": false
}
```

speaker_role

start_ts

end_ts

pii_redaction_flag

缺这些字段：对话审计失效，HITL 无法回听正确片段，高风险字段可能直接进入模型。

JSON 对照 3: video 不能只保留字幕

视频证据要同时能落到时间片段和关键帧。

BAD SAMPLE

```
{
  "asset_type": "video",
  "video_id": "VID-019",
  "subtitle": "Upgrade failed. Retry..."
}
```



GOOD SAMPLE · multimodal anchor

```
{
  "source_id": "support_video",
  "asset_type": "video",
  "segment_ts": "00:12:05-00:12:40",
  "frame_ts": "00:12:18",
  "transcript_ref": "SEG-778",
  "image_caption": "Firmware upgrade screen",
  "ocr_text": "Upgrade failed. Retry..."
}
```

segment_ts

frame_ts

image_caption

ocr_text

缺这些字段：关键帧证据链断裂，检索命中后也无法准确落到片段。

JSON 对照 4: ticket 必须携带租户、状态和时间

结构化数据也会漂；不是只有文档和媒体才需要 metadata。

BAD SAMPLE

```
{
  "asset_type": "ticket",
  "ticket_id": "T-1001",
  "status": "open"
}
```



GOOD SAMPLE · boundary + increment

```
{
  "source_id": "ticket_fact",
  "asset_type": "ticket",
  "source_system": "zendesk",
  "tenant_id": "tenant-a",
  "status": "open",
  "opened_at": "2026-04-14T08:00:00Z",
  "updated_at": "2026-04-14T09:00:00Z",
  "requester_role": "admin",
  "product_line": "edge-gateway"
}
```

tenant_id

updated_at

requester_role

product_line

缺这些字段：多租户过滤不稳、增量 ingest 不准、tool boundary 容易错路由。

PII 不只看字段，还要看动作

`contains_pii = true / false` 不能回答系统真正要做什么。

从“有没有 PII”升级为 “字段级分级 + 系统动作”

`store_raw`

能不能原文存储

`embed`

能不能原文索引 /
embedding

`retrieve`

能不能参与召回

`display`

能不能展示给用户

`pass_to_tool`

能不能传给工具动作

`human_review`

是否需要人工审核

一套实用的四级 PII 分法

等级不是为了贴标签，而是为了给默认动作提供入口。

public

公共帮助文档、公开错误码、公开教程

可索引、可检索、可展示

internal

内部 SOP、非个人化运维说明、内部流程

仅内部角色可用

sensitive

姓名、邮箱、手机号、客户评论、截图中的个人信息

默认脱敏后再进入检索 / 模型

restricted

token、密钥、支付信息、证件号、原始敏感音频

默认不进索引、不直送模型，必要时隔离或人工审核

字段 × 动作 × 场景：PII 不该只写成 true / false

矩阵化以后，系统才知道不同等级字段在不同动作上的默认边界。

等级	store_raw	embed	retrieve	display	pass_to_tool	human_review
public	允许	允许	允许	允许	允许	可选
internal	允许	条件允许	内部范围	条件允许	条件允许	建议
sensitive	允许但标注	脱敏后	条件允许	裁剪后	默认限制	建议
restricted	条件隔离	默认禁止	默认禁止	默认禁止	默认禁止	必须

落地要求：pii_policy_matrix_v1.csv 不是合规备注，而是后续 policy engine / gate / tool boundary 的动作映射。

先脱敏再入模，还是保真存储后查询裁剪

这不是绝对二选一，而是边界设计问题。

SAFER DEFAULT

先脱敏再入模

优点：风险更低，适合敏感字段进入检索/模型前的默认动作。

代价：信息损失可能影响检索和引用，尤其是时间、空间、上下文实体。

AUDITABLE

保真存储 + 查询裁剪

优点：证据链更完整，保留原始可复核能力。

代价：边界设计更复杂，必须有更强的 policy gate 和审计链路。

sensitive 优先脱敏后消费；**restricted** 默认不进入通用 serving。

行业新信号：为什么这件事现在更重要了

多模态、证据链和合规边界都在逼迫 metadata 从“备注”变成“接口”。

SIGNAL

Multimodal Retrieval

检索系统正在处理 text / image / video / audio; metadata filter 和多模态索引不再是可选项。

SIGNAL

Document Intelligence

文档解析强调 hierarchy、layout、bbox、provenance, 服务 citation 与 audit。

SIGNAL

PII De-identification

Presidio 等工具能辅助识别和匿名化, 但动作边界仍要由 policy matrix 明确。

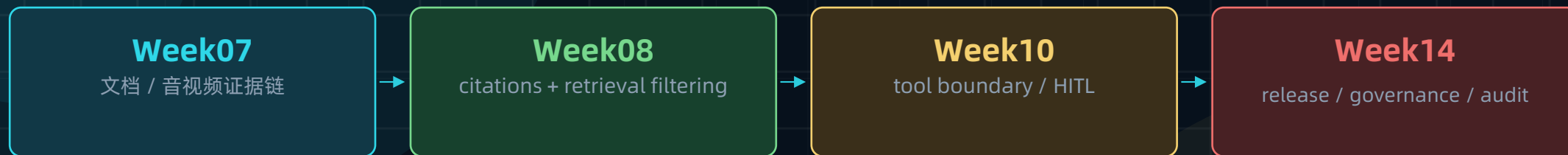
SIGNAL

AI Risk Governance

NIST GAI Profile 强调跨生命周期风险管理, privacy / information integrity / governance 要前置。

这节课和未来多模态系统的关系

今天写的 metadata，会一路被后面的证据链、RAG、工具和治理复用。



SYSTEM DESIGN

长期判断

最小 metadata 标准不是 Week02 的局部练习；它会变成整个 OmniSupport Copilot 从“可回答”走向“可执行、可回归、可审计”的公共语言。

直接动手：把标准和样例沉淀进 repo

本地演示只做一件事：创建可以被下一讲 contract gate 消费的规则文件。

TERMINAL · Week02 Lesson03 handoff

```
mkdir -p docs/blueprints/week02
mkdir -p tests/contract/fixtures/week02

touch docs/blueprints/week02/metadata_minimums_v1.md
touch docs/blueprints/week02/pii_policy_matrix_v1.csv
touch tests/contract/fixtures/week02/sample_records.json
```

[metadata_minimums_v1.md](#)[pii_policy_matrix_v1.csv](#)[sample_records.json](#)

TEACHING NOTE

不要一上来写 contract

先把 shared core、modality extension 和 PII 动作矩阵写清楚。

FIXTURE

至少 4 条好样本

分别覆盖 ticket / document / audio / video，让下一讲有真实 contract fixture。

NEGATIVE CASE

至少 1 条坏样本

故意保留风险字段或缺失锚点，用于 Lesson04 的 negative cases。

sample_records.json: 把四类资产收敛成 contract fixture

下一讲 Data Contract 能不能测试，取决于今天有没有准备好正反样例。

FIXTURE RECORD

document

good: 有 page_no / bbox / section_path
bad: 只有 text

FIXTURE RECORD

audio

good: 有 speaker_role / start_ts / end_ts
bad: 带手机号但没有 PII 动作

FIXTURE RECORD

video

good: 有 segment_ts / frame_ts / OCR
bad: 只有 subtitle

FIXTURE RECORD

ticket

good: 有 tenant_id / updated_at / product_line
bad: 只有 status

没有 negative cases 的 contract tests, 通常只能证明“样例能跑”，证明不了“系统能守门”。

本课自检与下一讲入口

从这一讲开始，系统终于知道一条输入到底能不能被引用、送模、展示和追责。

Self-check

- 我知道 metadata 不是备注，而是系统接口
- 我能说出 shared core + modality extension 的统一方法
- 我已经为 ticket / document / audio / video 准备最小 metadata 清单
- 我知道 page_no / bbox / section_path / speaker_role / start_ts / frame_ts 为什么是硬约束
- 我已经把 PII 写成字段 × 动作矩阵，而不是 true / false
- 我已经准备好 metadata_minimums_v1.md、pii_policy_matrix_v1.csv、sample_records.json

NEXT · L4

下一讲：把 Data Contract 做成工程门禁

今天的 metadata 和 sample_records 会进入：

- JSON contracts
- contract tests
- compatibility check
- accept / warn / quarantine / reject

Lesson04 才有东西可守。

把 Data Contract 做成工程门禁

Schema、语义、质量、兼容性：从“字段说明”升级为可执行的输入准入系统

核心判断：Contract 只有能放行、拦截、报警并进入 CI/CD，才算系统能力。

Shape / Schema

Semantics

Metadata / Evidence

Policy / Access / PII

Quality / SLA

gate

本课不是讲“更长的字段表”

讲完之后，学生应该知道一份 contract 什么时候才算能进系统。

- 01 —— 拆对象 Schema / Contract / Manifest / Policy 彻底分开
- 02 —— 立五层 Shape、Semantics、Evidence、Policy、Quality
- 03 —— 解剖两份 ticket_contract 与 doc_asset_contract 的真实门禁
- 04 —— 判断兼容 additive / conditional / breaking 不能只靠字段 diff
- 05 —— 进工程链 fixture、pytest、manifest binding、seed loader dry-run

本课产出

- 四类 JSON contract 复核
- week02 fixture 正/反例
- 兼容性判断记录
- contract test 结果解读

Week02 到这里：从输入认知进入输入工程

课时4是 hub：它把资产清单、元数据和 PII 策略变成机器可读的 gate。



L4 不是独立知识点，而是 Week02 所有输入控制面的工程“连接器”。

Schema 只回答“像不像” Contract 才回答“够不够格进系统”

能校验

能拦截

能报警

能进 CI/CD

能讨论兼容性

真正的生产问题

字段常常还在，类型也没错；真正危险的是语义、证据链、权限和质量标准静默漂移。

先看学生讲义页的总图

Lesson04 是 hub：下游 contract tests、RAG response contract、tool boundary、audit release 都依赖它。



先把 4 个对象彻底分开

否则后面所有讨论都会混：schema、contract、manifest、policy 各自回答的问题不同。

JSON Schema

像不像

字段形状、类型、枚举、格式是否合法

Data Contract

够不够格

shape + semantics + evidence + policy + quality 是否一起构成准入标准

Manifest

怎么接

本次 ingest 准备接哪一批数据，按什么模式接

Policy

谁能碰

哪些字段、动作、角色受限制；如何脱敏、拦截或升级审查

一句话：Schema 看长相，Contract 看资格，Manifest 看批次，Policy 看边界。

为什么 JSON Schema 不等于 Data Contract

Schema 是必要底座，但不足以承担生产准入。

对比项	JSON Schema 更擅长	Data Contract 还必须回答
结构	字段、类型、枚举、required	这些结构在业务上意味着什么
合法性	记录是否“长得像”目标对象	是否足够安全且足够可用
兼容性	结构变化有没有破坏校验	是否会带偏检索、生成、工具动作
运行时门禁	能否做基础 schema 验证	什么时候放行、隔离、拒收
审计责任	能力有限	owner、policy、PII、release、run evidence 闭环

关键判断：schema 能过，不代表系统安全

为什么付费课程里必须把 Contract 讲厚

生产中最常见的事故，不是字段直接消失，而是字段“看起来还在”。

字段没错，但意义变了

status 仍是 string，但生命周期语义漂移；
updated_at 仍是 timestamp，但变成 ETL 时间。

文档还在，但证据链断了

page_no / section_path / bbox 丢失后，
citation、audit、entitlement 一起失效。

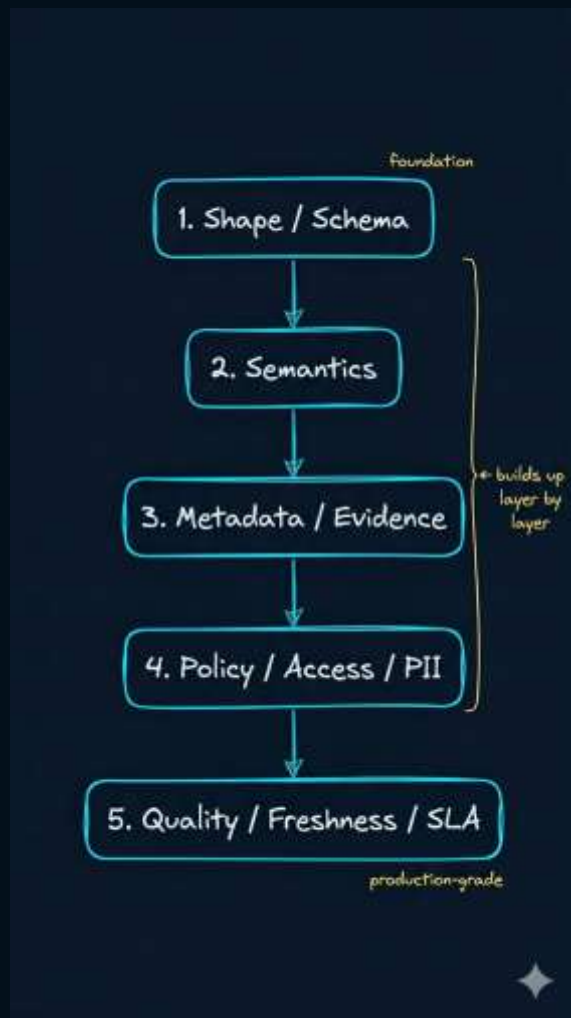
权限字段还在，但没变成 gate

access_scope / pii_level 只是列，不进入检索与生成路径，就无法阻止越权消费。

如果 contract 不能影响运行时动作，它就只是“文档”。

一份真正可执行的 Contract, 至少要有 5 层

把 contract 当成“分层防线”，不是当成更长的字段表。



1 Shape / Schema

对象形状是否完整

2 Semantics

字段意义是否稳定

3 Metadata / Evidence

是否能引用与追责

4 Policy / Access / PII

谁能碰、碰到哪一步

5 Quality / Freshness / SLA

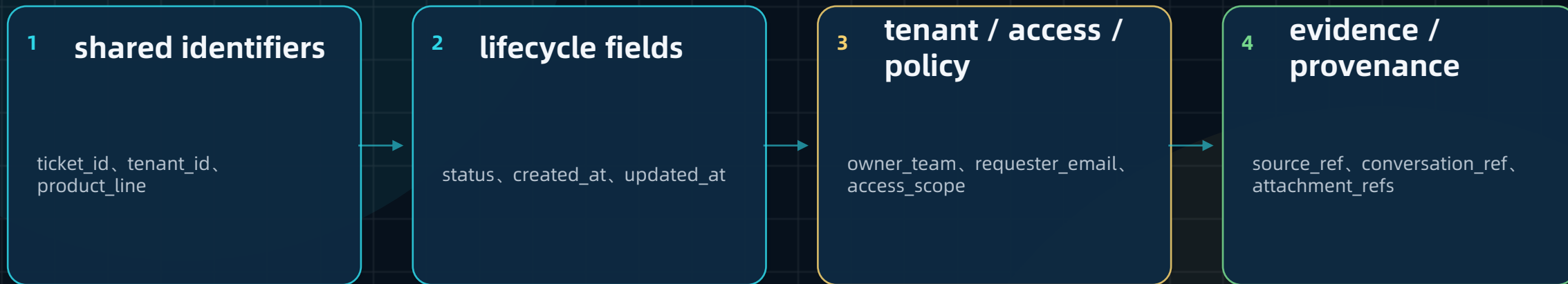
现在是否够格进入系统

五层不是目录，而是五个 Gate Question

层	ticket 示例	document 示例	gate 会问什么
Shape	status 是否存在且为 string	page_no 是否为 int	结构是否完整
Semantics	updated_at 代表什么时间	doc_version 是否正式版本	语义是否漂移
Evidence	ticket_id / tenant_id	page_no / section_path / bbox	能否引用与追责
Policy	requester_email 是否 mask	license_tag 是否允许分发	是否越权 / 违规
Quality	枚举违规率、新鲜度	版本延迟、缺页率	放行、警告还是拦截

把“描述字段”变成“定义动作”

真实解剖一：ticket_contract.json 先看什么



repo path

contracts/data/ticket_contract.json

先不要新造 contract
先按四组字段复核现有契约

ticket contract 守的不是“字段漂亮”，而是工单动作、指标、权限和追责不跑偏。

ticket_contract 至少要守住什么

模块	典型字段	它为什么关键	坏了会怎样
身份定位	ticket_id, tenant_id, product_line	确定多租户与产品线边界	越权 / 混租户
生命周期	status, created_at, updated_at	KPI、优先级、升级逻辑依赖	统计与路由一起偏
责任边界	owner_team, requester_email, access_scope	谁负责、谁能看、谁能执行	泄露 / 误操作
证据链	source_ref, conversation_ref, attachment_refs	后续引用与审计可落地	无法复盘

ticket gate = action boundary + audit boundary

ticket_contract 最常见的两个失败

失败 1：枚举扩展未收敛

- 表面：`status` 还是字符串，schema 可能继续通过
- 实际：路由、统计、工具动作一起偏
- 默认：conditional，必须 review 下游消费方

失败 2：时间字段语义漂移

- 表面：`updated\_at` 仍然存在
- 实际：增量窗口和 SLA 监控一起失真
- 默认：breaking，至少需要迁移窗口

语义漂移，即使 schema 不变，也可以是 breaking。

真实解剖二：doc_asset_contract.json

doc_version

无法区分当前有效版本

page_no

引用页码直接失效

section_path

上下文层级没了

bbox

图文定位和证据截图崩掉

license_tag

分发和二次使用边界不清

文档资产没有 locator，就没有可信 citation。

document locator

contracts/data/doc_asset_contract.json

required locator:

doc_version
page_no
section_path
bbox
license_tag

如果这些字段丢了, citation / audit / entitlement 会怎么坏

丢失字段	citation 会怎么坏	audit 会怎么坏	entitlement 会怎么坏
page_no	无法回指页码	无法证明引用来自哪页	授权页范围无法证明
section_path	章节上下文无法恢复	找不到原始章节结构	章节级授权会失真
bbox	图表 / 截图无法稳定圈定	证据截图难复核	局部区域是否可展示难判
license_tag	不能区分可引用与不可分发	无法说明为什么允许输出	分发边界直接失守
access_scope	可能混入不该暴露段落	无法还原授权链条	检索与生成都可能越权

兼容性不是一句“升级到 v2”就够了

兼容性不是技术问题，而是下游承诺问题。

additive

新增可选字段
如 priority_label

通常可放行，但要记录

conditional

扩充 status 枚举
收紧 freshness 阈值

需要 review 下游是否准备好

breaking

删除 / 重命名字段
语义漂移 / locator 改义

默认拦截或给迁移窗口

不要在“全放行”和“全拦截”之间摇摆

三个最值得练的变更案例

变更案例	为什么不是一眼判断	推荐级别	建议动作
ticket.status 新增 in_progress	结构没坏，但会穿透统计与 tool route	conditional	review 路由 / 指标 / UI
document.doc_version 从 optional 改 required	老数据可能直接不再合法	breaking	迁移窗口 + fixture 回归
audio.speaker_role 枚举从 4 个缩成 2 个	老转写样本会失配，归因语义会塌	breaking	拦截 + 补充映射规则

底层支持 Schema Evolution, 上层反而更需要 Gate

底层 schema evolution

- 能否追加、重命名、改变类型或分区
- 关注存储兼容、元数据版本、是否重写数据
- 目标：让表结构变化更平滑

上层 contract gate

- 业务语义、权限边界、引用能力是否还成立
- 检索、生成、工具动作是否会被带偏
- 目标：让 AI 消费链路可负责

能存下来 ≠ 能安全用起来

这套 Gate 真正怎么进入工程链



contract draft → repo → fixture → tests → manifest binding → dry-run → gate

跑 `pytest tests/contract/ -v` 时，到底在验证什么

统一命令

```
docker compose --profile tools \
  --env-file infra/env/.env.local \
  -f infra/docker-compose.yml run --rm devbox \
  pytest tests/contract/ -v
```

你至少应该看到

- `test_json_schemas.py` 被执行
- `contract schema` 成功加载
- 故意改坏 `enum` / `required` / `type` 会失败

你在看什么	它说明什么
JSON schema 是否能加载	contract 本身结构是否合法
fixtures 是否满足当前 schema	课程最小样例是否还站得住
manifest schema 是否稳定	后续 loader 是否还有统一入口

如何读 contract test 失败

失败现象	你先怀疑什么	讲师点评
required 缺失	fixture 漏字段, 或 contract required 过严	先看这是历史数据问题还是契约升级问题
enum 不匹配	上游枚举漂移, 或 fixture 已经过时	不要立刻放宽 enum, 要先确认下游动作
format 错误	contract 太松 / 太严, 或样例是脏数据	标准不是为了过测试, 而是为了可消费
compatibility diff 报 breaking	下游承诺被打破	不能只看技术上还能不能跑

读失败 = 读输入系统的边界变化

行业新信号：Contract 正在变成工程门禁

ODCS / Data Product

契约不只 schema，也纳入 quality、roles、SLA、team 等结构化部分。

CLI / CI / Fixtures

成熟团队用 lint、test、compatibility check 管 contract，而非人工巡检 Wiki。

Schema Evolution + Gate

底层演进能力增强后，上层 compatibility policy 更不能省。

Lineage / Run Evidence

contract test 与后续 manifest / loader / run evidence 连接，形成可追溯闭环。

直接动手：把这节课落到 repo 里

1 先看 contract

ticket / doc / audio / video 四类 JSON contract

2 统一 fixture

tests/contract/fixtures/week02 正例、反例、变更记录

3 跑 contract tests

Docker devbox 执行 `pytest tests/contract/ -v`

4 记录兼容性

至少一条 conditional + 一条 breaking 判断

fixture path

```
mkdir -p tests/contract/fixtures/week02
# positive samples / negative samples / compatibility record
```

检查表：5 分钟复核一份 Contract

Shape

required / type / enum 是否覆盖最小可消费结构？

Semantics

关键字段的业务含义是否写清并能被 review？

Evidence

locator / provenance / citation 需要的字段是否强制保留？

Policy

PII、access_scope、license_tag 是否进入可执行门禁？

Quality

缺字段、枚举漂移、新鲜度不达标时动作是什么？

Compatibility

新增、删除、重命名、语义漂移如何分级？

不要问“测试能不能过”，要问“下游承诺还成不成立”。

本课最容易误解的 5 件事

误解

正确理解

contract 就是字段表

contract 是输入门禁

schema 能过就说明系统没问题

语义漂移一样会把系统带偏

兼容性只看字段是否存在

还要看枚举、语义、policy、evidence

pytest 只是形式化检查

它是 Week02 进入持续工程链的第一步

Data Contract 讲完就结束了

下节 manifest 和 seed loader 会继续消费它

小结： Week02 的输入控制面第一次具备工程可执行性



记住 1

schema 是底座，不是全部答案

记住 2

semantic / evidence / policy / quality 才让 contract 能做 gate

记住 3

compatibility 是消费方承诺，不只是字段变化

下一讲：契约怎样驱动一次 ingest 批次

Lesson05 会把 contract 接到 source manifest、load mode、gate action 和 run evidence。

source manifest

本次接谁、从哪里读、按什么窗口

load mode

full / incremental / cdc / replay / backfill

gate action

accept / warn / quarantine / reject

run evidence

把本次运行变成 Week03 的起跑线

课后最小行动：复核四类 JSON contract + 准备 fixture + 跑一次 contract tests

契约驱动的采集策略

Manifest、增量窗口、拦截与 Week03 起跑线

把“规则文件”推进到运行时：从 contract 选择、schema check、seed_loader dry-run，到 accept / quarantine / reject / warn 与 run evidence。



Week02 的真正收官：不是写完 contract，而是让 contract 驱动 ingest admission



Contract

定义“什么样的数据合格”
但它不是本次采集计划。

Manifest

定义“这次到底接哪一批”
并绑定窗口、模式与责任人。

Run evidence

把本次判断留下来
供 Week03 replay / backfill / state 使用。

这节课解决的问题：系统还没有真正“跑起来”

前三课完成了规则，但还缺“本次运行”的输入控制面。

Week02 前四讲已经完成

已完成

资产值得进入系统

已完成

metadata / PII 最低标准

已完成

机器可读 Data Contract

课时5必须补上的控制面

缺口 1

这次 ingest 的批次是谁？

缺口 2

全量、增量、CDC，还是回放 / 补数？

缺口 3

字段缺失、枚举漂移、PII 不完整时怎么办？

缺口 4

结果如何留下证据，接到 Week03？

关键词：runtime input control plane

本课完成什么：5 个能力 + 7 个工件

每一项都要能接到后续 Week03 的 ingest / replay / backfill。

能力 1

解释为什么 Data Contract 不是 ingest plan

能力 2

用 JSON manifest 声明一次 ingest 批次

能力 3

区分 full_snapshot / incremental_cursor / cdc / replay / backfill

能力 4

设计 accept / quarantine / reject / warn 四类门禁动作

能力 5

跑通 source_manifest_schema + manifest + seed_loader dry-run

产出工件

source_manifest_schema.json

manifest_tickets_synthetic_v1.json

manifest_edge_gateway_pdf_v1.json

manifest_workspace_helpcenter_v1.json

manifest_week02_practice_v1.json

ingest_strategy_v1.md

seed_loader dry-run 观察记录

Week02 位置图：从输入地图到 ingest 起跑线

课时5是把前面所有规则串进运行时的桥。



今天的关键切口

contract 不是 ingest plan; manifest 不是文件清单;
gate 不是二元 pass/fail。

今天的最终交付

一个可信的 ingest baseline：可解释、可重跑、可追踪。

先看一张总图：规则连成准入链



contract defined here

contract executed here

Week03 consumes this baseline

现在真正要区分的 4 个对象

很多生产问题，都是因为把 contract、manifest、ingest plan 混成一件事。



记忆法：世界地图 → 合格标准 → 本次装车清单 → 通关结果

一句话记住它们：不要把“标准”和“本次运行”混在一起

这页用于把抽象对象压成学生能带走的心智模型。

Inventory

世界地图

说明世界里有哪些 route 和 asset

Contract

合格标准

说明什么样的记录 / 元数据能进系统

Manifest

本次装车单

说明这一次从哪里接、按什么窗口接

Run evidence

通关结果

说明这次通过、隔离、拒收、警告了什么

Manifest 不是文件清单，而是运行时意图声明

只列路径不会留下 ingest 语义，也不能复现一次运行。

把 manifest 当成文件清单

只能列路径

窗口与模式缺失

contract 绑定缺失

owner / release 不可追

失败后不知道该 replay 还是 backfill

把 manifest 当成运行时意图

绑定 contract

声明 load_mode

定义 window / cursor / snapshot

携带 owner / pii_level / release_id

为 dry-run 与 run evidence 留下锚点

结论：路径只是 location，manifest 要表达本次 ingest 的语义

这周的实践格式：围绕 repo 已存在的 JSON manifest 体系

不要再把 YAML manifest 当作 Week02 主实践格式；本课对齐当前仓库 JSON 文件。

schema data/seed_manifests/source_manifest_schema.json

tickets data/seed_manifests/manifest_tickets_synthetic_v1.json

edge gateway pdf data/seed_manifests/manifest_edge_gateway_pdf_v1.json

helpcenter data/seed_manifests/manifest_workspace_helpcenter_v1.json

practice data/seed_manifests/manifest_week02_practice_v1.json

课堂提示

先看 schema，再看 3 个现有 manifest，最后再补 practice manifest。顺序不能倒。

Manifest anatomy: 每组字段各守一道门

好的 manifest 不是越长越好，而是每组字段都有运行时用途。

字段组	它守的门	典型字段
source identity	这批数据到底是谁	source_id / asset_type / owner
location	系统去哪里读它	path / uri / bucket / prefix
contract binding	它应该服从哪份 gate	contract_ref
load semantics	这次是怎么接的	load_mode / cursor / window / snapshot
policy context	运行时边界和版本	pii_level / access_scope / release_id
evidence context	以后怎么追	generated_at / manifest_version / notes

少掉 location 是脚本硬编码；少掉 contract_ref 是没有统一 gate；少掉 evidence context 是以后不可追。

一份最小练习版 JSON manifest: 只保留课堂必须讲透的字段

示例来自学生讲义页并做了投屏友好的缩短。

JSON

```
{
  "manifest_version": "v1",
  "release_id": "week02-core-r1",
  "sources": [{
    "source_id": "ticket_core_daily",
    "asset_type": "ticket",
    "path": ".../manifest_tickets_synthetic_v1.json",
    "contract_ref": "contracts/data/ticket_contract.json",
    "load_mode": "incremental_cursor",
    "cursor_field": "updated_at",
    "window_start": "2026-04-13T00:00:00Z",
    "window_end": "2026-04-14T00:00:00Z",
    "owner": "support-ops",
    "pii_level": "restricted"
  }]
}
```

contract_ref

把本批次接到正确 gate

load_mode + window

说明这次和上一批的关系

owner + pii_level + release_id

让责任、风险、版本可追

Contract 约束记录长什么样; Manifest 约束批次怎么进来;
Loader / gate 决定现在能不能进。

manifest 字段组如何被 loader / gate 消费

字段只有被运行时消费，才不是“文档字段”。



五种采集模式不要死记：背后是五种批次关系

先讲前 3 种：完整世界、变化窗口、事件流。

full_snapshot

manifest 至少声明: snapshot_date

某个时点的完整世界

风险：成本高、重复索引

incremental_cursor

manifest 至少声明: cursor_field +
window_*

最近变化过的对象

风险：漏数、重数、时区错误

cdc

manifest 至少声明: checkpoint_field /
cdc_cursor

关心事件流，而不是静态表

风险：配置复杂、补数策略难

replay / backfill: 不是新数据，而是修复历史运行关系

这两种模式一定要讲清，否则 Week03 最容易做成“重复写入”。

replay

不是新数据；是重跑旧批次。

至少声明 `replay_from_manifest / replay_reason`。

风险：重复写入、版本混淆。

backfill

不是在线变化；是补历史空洞。

至少声明 `backfill_range / reason`。

风险：影响范围过大、资源争抢。

别背模式，先问 3 个问题

Q1

这一批和上一批到底是什么关系？

Q2

这一次覆盖完整世界，还是只接变化部分？

Q3

如果失败，后面应该重放、补数，还是继续追增量？

生产里的坑：load mode 写错，后面全链路都会误判

这页体现实战经验：不是 manifest 语法问题，而是批次语义问题。

timezone drift

window_start / window_end 没有统一时区，导致漏数或重数

cursor ambiguity

cursor_field 不稳定，updated_at 被回写或延迟写入

duplicate writes

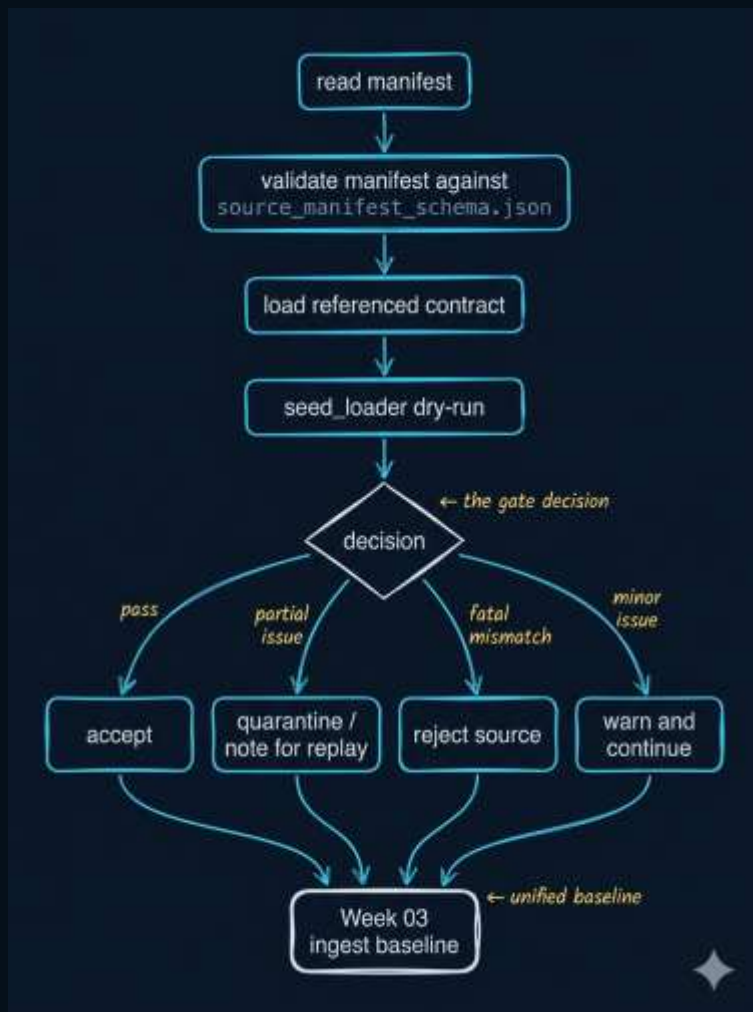
replay 没有 idempotency key，重跑变成重复写

missing state

没有 release_id / source_id / manifest_version，无法定位哪一轮坏了

先把“这一次的边界”写清楚，再谈性能、并发、吞吐。

运行时门禁不是二元判断，而是 4 类动作



accept

合格进入下一层

quarantine

部分问题，隔离并记录待 replay

reject

严重错误，整源拒收

warn

可容忍问题，放行但必须记日志

四类动作：不是结果标签，而是后续处理路径

门禁判断必须告诉系统下一步怎么做。

动作	系统行为	什么时候用
accept	记录合格，可以进入下一层	字段 / 元数据 / PII / 枚举全部通过
quarantine	隔离，暂不进主链路	个别记录坏了，但整批还有可用部分
reject	整个 source 不接收	manifest 严重错误、contract 不匹配、关键字段缺失
warn	暂时放行，但必须记日志	非关键描述字段缺失、可容忍延迟

accept → ingest baseline

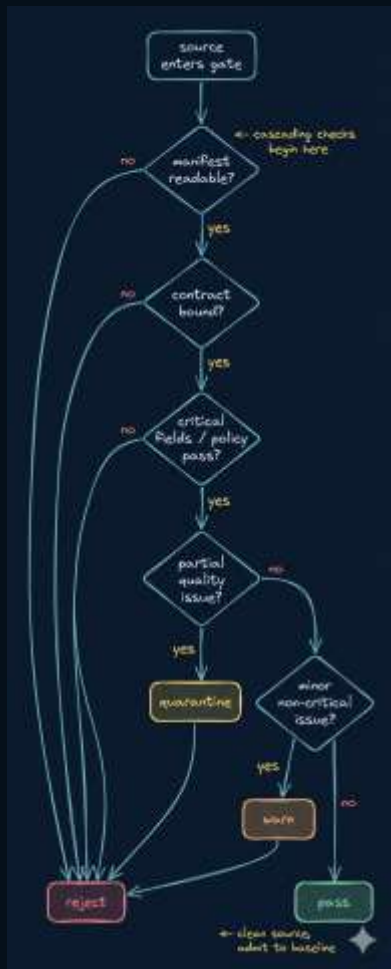
quarantine → patch / replay

reject → fix admission first

warn → observability debt

完整决策树：Week02 学的是分层处理问题

目标不是把所有坏数据拒之门外，而是按严重程度把问题分流。



1 manifest readable?

没有 manifest，直接 reject，不进入后续讨论。

2 contract bound?

没有 contract_ref，不能执行统一 gate。

3 critical fields / policy pass?

关键字段或 PII policy 失败，不能假装合格。

4 partial quality issue?

局部问题优先 quarantine，为后续 patch / replay 留空间。

5 minor non-critical issue?

非关键问题可以 warn，但要进入 run evidence。

clean source → pass → admit to baseline

一次 dry-run 报告到底应该怎么读

不要只看“有没有报错”，要读出 4 层运行证据。

01

哪个 manifest 被读取

本次批次边界是否明确

02

它引用了哪份 contract

source 和准入标准是否闭环

03

哪些 source 被 accept / quarantine / reject / warn

运行时门禁是否开始发挥作用

04

有没有 release_id / source_id / owner

后续追踪和 Week03 衔接是否可能

典型阅读顺序：schema 通过 → contract 绑定 → gate judgment → run evidence 可沉淀

dry-run 报告怎么读，才能接上 Week03

Week03 不是从零开始写 ingest runtime，而是消费 Week02 的准入结果。

报告里看到什么	Week03 如何继续消费
manifest 被读到	批次存在性成立，state 有起点
contract 绑定成功	admission 前提成立，gate 可复用
source 被 quarantine	需要 patch、replay 或补 metadata
source 被 reject	不能进主链，必须先修准入
window 声明不清	state / watermark 无法建立

关键判断

Week03 的 state、watermark、idempotency，不是凭空长出来的；它们直接来自 manifest 的 load_mode、contract 的 shape、gate 的分流动作。

直接动手前：先按这个顺序打开文件

录播时这一页用于从 PPT 平滑切到终端。

TEXT

```
data/seed_manifests/source_manifest_schema.json
```

```
data/seed_manifests/manifest_tickets_synthetic_v1.json
```

```
data/seed_manifests/manifest_edge_gateway_pdf_v1.json
```

```
data/seed_manifests/manifest_workspace_helpcenter_v1.json
```

闭环问题

source_id / asset_type / contract_ref 有没有闭环?

模式问题

load_mode 需不需要 cursor_field / window_* / snapshot_date?

责任问题

owner / release / pii_level 是否写清?

Docker-first: 统一用 devbox 跑 seed_loader dry-run

用同一个容器环境跑，避免“你本机能跑、我本机不能跑”。

BASH

```
touch data/seed_manifests/manifest_week02_practice_v1.json
touch docs/blueprints/week02/ingest_strategy_v1.md

docker compose --profile tools --env-file infra/env/.env.local \
  -f infra/docker-compose.yml run --rm devbox \
  python -m pipelines.ingestion.seed_loader \
  --manifest-dir data/seed_manifests
```

读取了哪些 manifest

批次存在性

哪些 source 批次语义不清

窗口/模式问题

哪些 source 被 contract / metadata / PII 拦截

准入门禁开始工作

dry-run 的价值：先判断、先记录、先暴露输入风险，再进入 Week03 真正采集。

run evidence: 这次运行必须留下“以后能追”的证据

运行记录不是日志堆积，而是后续排障、回放、发布、审计的入口。

identity

run_id / release_id / manifest_version / git_sha

source linkage

source_id / asset_type / contract_ref / owner

gate result

accept / quarantine / reject / warn + reason_code

window state

load_mode / cursor / window_start / window_end / snapshot_date

next action

patch / replay / backfill / continue incremental

经验判断：没有 run evidence 的 dry-run，只是“看起来跑过一次”。

为什么 Week02 不直接把 Week03 也做完？

这是工程边界，不是课程没讲完。

Week02 要解决什么	Week03 再解决什么
输入资产和值得接入的 source	真正的 batch / incremental ingestion 实现
metadata / PII 最低标准	幂等、补数、回放、失败恢复
contract + manifest 的准入门槛	真正的数据搬运与入湖
dry-run 与 run evidence 起点	运行时调度与长期运维

Week02 结束时拿到的不是“完整 ingest 系统”，而是一个可信的 ingest baseline。

本课最容易误解的 5 件事

每个误解背后，都是一个真实生产系统会出事的点。

误解	正确理解
manifest 只是列几个文件路径	manifest 是一次 ingest 批次的运行时声明
只要 contract 正确，manifest 随便写	manifest 决定这一批怎么进来
dry-run 通过就说明 Week03 完成	dry-run 只是起跑线，不是终局
quarantine 和 reject 差不多	quarantine 可隔离观察；reject 整批不能接
Week02 应该把 ingest 做完	Week02 要建 baseline 与 run evidence

行业信号：数据工程正在把“运行意图”和“证据链”对象化

这页用于体现技术前沿，但不要变成工具介绍会。

Data Contract 标准化

ODCS 与 OpenMetadata 都强调 schema、语义、质量、安全、SLA 等机器可读契约。

Orchestrator 资产化

Dagster asset checks / partitions / backfills 把质量、分区、历史补数放进资产运行语义。

Lineage / run evidence

OpenLineage 用 facets 给 Run / Job / Dataset 挂载上下文，适合承接 run evidence。

Freshness / state-aware

dbt freshness 用 warn_after / error_after 表达数据新鲜度阈值，体现“时间窗口也是契约”。

未来的采集配置不应该是脚本参数，而应该是可验证、可追踪、可复现的运行时对象。

课后最小行动：用 3 步把 Week02 真正收口

学生做完这 3 件事，才算拿到 Week03 起跑线。

01 阅读 schema + 3 个现有 manifest

先理解仓库里的 JSON manifest 约定。

02 创建或补齐 manifest_week02_practice_v1.json

明确 source、contract、load_mode、window、owner、pii_level。

03 跑一次 Docker devbox seed_loader dry-run

写下 accept / quarantine / reject / warn 的门禁判断。

提交物不是“截图证明跑过”，而是能解释本次 gate judgment 与下一步处理路径。

小结：四件事连起来，Week03 才有稳定起跑线

下一步不是再讨论 manifest 理论，而是把 Week02 工件真正跑进实验和 Week03。



只有这四件事连起来，Week03 才不是“写采集脚本”，而是在一个可验证、可追踪、可回放的输入基线上继续建设。